# Kurukshetra  Engine Kart

## Full-Vehicle Telemetry, Software & Digital Twin System

*Hardware Plan · Software Architecture · Digital Twin Specification*

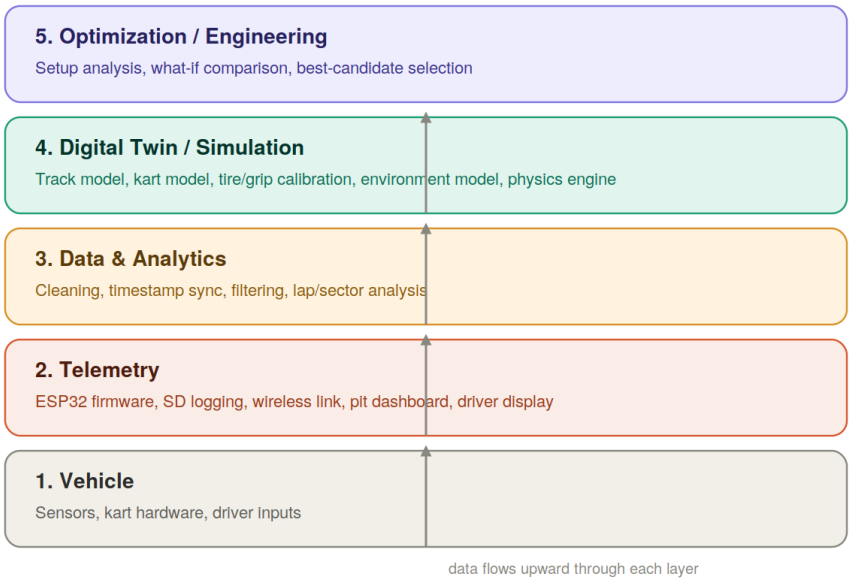
| Field | Detail |
|---|---|
| Prepared by | Innovation |
| Team | Kurukshetra Racing, SKCET |
| Category | EKVC Season 4 — Innovation Round |
| Scope | V1 core build + V2/V3 roadmap + Digital Twin architecture |

# 1. Project Overview

This document defines the complete engineering plan for the Kurukshetra Kart telemetry and digital twin system: which sensors to install, how the embedded and pit software is structured, and how collected data becomes a validated simulation model. It is organized in three parts — Hardware, Software, and Digital Twin — so that the embedded team, the pit/dashboard team, and the analytics/simulation team can work from the same document without stepping on each other.

The guiding principle throughout: collect real measurements first, derive a calibrated model second, and only then use that model to make predictions. A simulator that hasn't been checked against real telemetry is a guess with graphics.

## 1.1 System Summary




*The system is organized into five layers. Data flows upward: raw vehicle signals become telemetry, telemetry becomes analytics, analytics feeds the digital twin, and the digital twin supports optimization.*

# 2. Hardware & Sensor Architecture

Not every measurable quantity needs a dedicated sensor in V1. Some values are calculated from others; some are only useful once the basic system is working reliably. The sensor list below is staged across three versions so the team builds a working core system first.

## 2.1 Core Sensors — V1 (Essential)

| # | Parameter | Sensor | Why it's needed | Priority |
|---|---|---|---|---|
| 1 | Wheel speed | Hall-effect sensor + magnet | Vehicle speed, wheel behaviour, distance | Essential |
| 2 | Engine RPM | Inductive pickup / tach sensor | Engine speed, gear analysis, powertrain model | Essential |
| 3 | Engine temperature | DS18B20 / temperature probe | Engine thermal monitoring | Essential |
| 4 | Steering angle | Rotary potentiometer / encoder | Steering input, corner analysis | Essential |
| 5 | IMU / G-force | MPU6050 / better IMU | Acceleration, braking, cornering, yaw | Essential |
| 6 | Obstruction distance | HC-SR04 (V1) / better ToF sensor (later) | Distance, closing speed, time-to-contact | Essential |
| 7 | Throttle position | Potentiometer / Hall throttle sensor | Driver input, vehicle model | Strongly recommended |
| 8 | Brake position/pressure | Brake pressure or position sensor | Braking analysis, driver model | Strongly recommended |

## 2.2 Wheel-Speed Sensing — Sensor Count Options

| Option | Sensors | What you get |
|---|---|---|
| A — Minimum | 1, on rear driven wheel | Basic vehicle speed — good enough for V1 |
| B — Recommended | 2, left + right rear wheels | Detects difference between wheels; better cornering data |
| C — Full | 4, one per wheel | Individual wheel speed, slip estimation, differential behaviour, full cornering analysis |

*Do not make four-wheel sensing mandatory for V1 — start with Option A or B and expand later.*

## 2.3 Engine RPM — Critical Safety Note

**Important**

Do not connect the ESP32 directly to a spark-plug ignition/high-voltage line. Use proper signal conditioning and isolation. The engine bay is electrically hostile — ignition noise and voltage spikes can destroy a microcontroller quickly without isolation components in between.

## 2.4 What to Buy First — V1 Parts List

| Sensor | Qty | Purpose | Est. Cost (₹) |
|---|---|---|---|
| ESP32 Dev Board | 1 | Central controller | 400–600 |
| Hall-effect wheel sensor | 1–2 | Wheel speed | 80–120 each |
| RPM pickup (inductive/Hall) | 1 | Engine RPM | 150–300 |
| DS18B20 | 1 | Engine temperature | 80–120 |

| Sensor | Qty | Purpose | Est. Cost (₹) |
|---|---|---|---|
| Rotary potentiometer | 1 | Steering angle | 50–100 |
| MPU6050 | 1 | G-force + gyro | 120–200 |
| HC-SR04 | 1 | Obstruction / TTC | 80–150 |
| Throttle position sensor | 1 | Driver input | 100–200 |
| BME280 | 1 | Ambient temp / humidity / pressure | 150–250 |
| MicroSD module + card | 1 | Full dataset logging | 150–250 |
| OLED display (0.96" I2C) | 1 | Driver display | 180–250 |
| RF/Bluetooth module (nRF24L01 or similar) | 1 | Pit wireless link | 150–300 |
| Voltage regulator + isolation components | 1 set | Clean, isolated power | 200–350 |
| Wiring, connectors, shielding, mounts | — | Reliable install | 500–800 |

**Estimated V1 total: approx. ₹2,300–4,000 — fits within a ₹6,000 budget with headroom.**

## 2.5 Staged Expansion — V2 and V3

### V2 additions

- Brake pressure sensor
- GPS / GNSS module
- Gear position sensor
- Second wheel-speed sensor (if starting from Option A)
- Higher-quality IMU (MPU6500 / ICM-42688)
- Better distance/obstruction sensor (ToF)

### V3 — Digital Twin-grade additions

- Four-wheel speed sensing
- Tire temperature (IR sensors, per wheel)
- Tire pressure (per wheel)
- Suspension travel (linear potentiometer/encoder, per wheel)
- More accurate GNSS
- Additional drivetrain measurements

*Suspension travel and tire sensing are genuinely complex to mount reliably — keep these out of V1 and V2 entirely.*

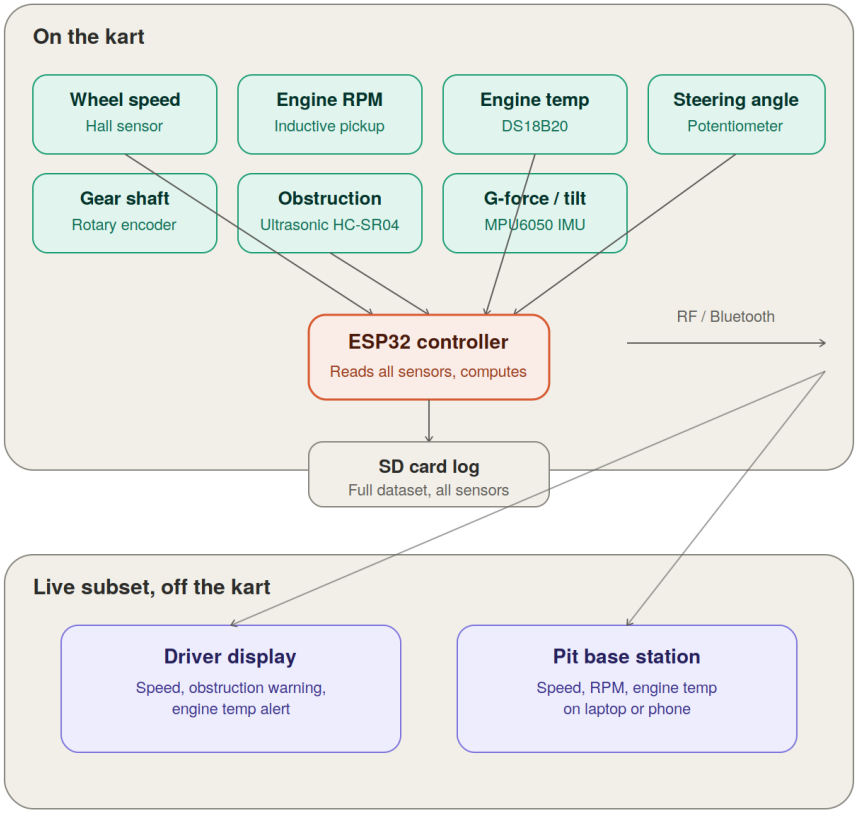
## 2.6 The 10 Signals That Matter Most

If asked "what data actually matters?", this is the answer — these ten give the foundation for understanding what the kart did; everything else helps explain why.

| # | Signal |
|---|---|
| 1 | Speed |
| 2 | Engine RPM |
| 3 | Throttle |
| 4 | Brake |

| # | Signal |
|---|---|
| 5 | Steering angle |
| 6 | Longitudinal acceleration |
| 7 | Lateral acceleration |
| 8 | Yaw rate |
| 9 | Engine temperature |
| 10 | Track position |

## 2.7 Hardware Data Flow




*Sensors feed the ESP32 controller. The full dataset logs to SD card; a filtered live subset streams wirelessly to the driver display and pit base station.*

# 3. Software Architecture

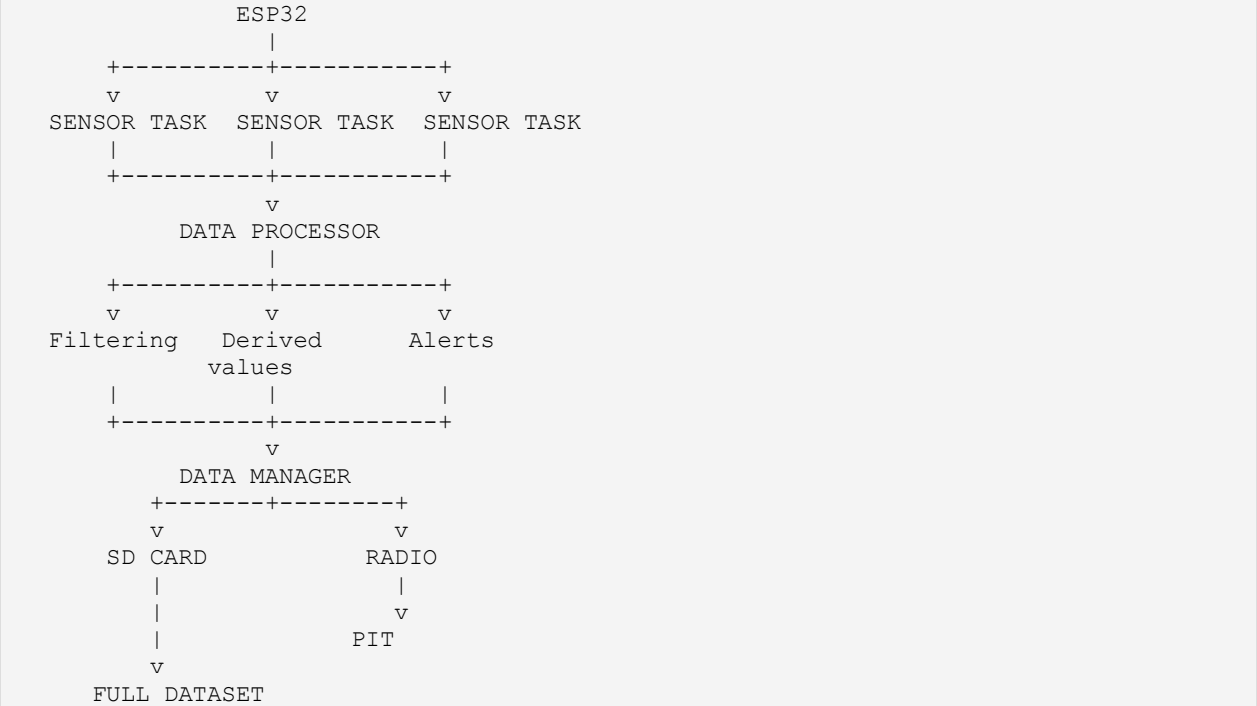
The software is organized into five layers, each with a distinct responsibility. This separation matters because it lets different subteams (embedded, pit/dashboard, analytics, simulation) build and test independently.

## 3.1 Embedded Firmware (Layer 1–2)

Runs directly on the ESP32. Responsibilities:

- Sensor acquisition
- Filtering
- Timestamping
- Derived calculations (e.g. speed from wheel pulses, time-to-contact from distance)
- Data logging to SD card
- Driving the steering-mounted display
- Sending wireless packets to the pit
- Fault detection

### Firmware task structure




> The wireless system must never control the primary logging system. If radio communication fails mid-run, sensors and SD logging continue — the run's data is never lost because of a wireless dropout.

### Data record format

Every telemetry record carries a timestamp. Example structure (for reference — not stored as JSON on the ESP32 itself):

```
{
  "timestamp": 12.450,
  "speed": 58.3,
  "rpm": 7120,
  "gear": 4,
  "steering": 13.4,
  "throttle": 82,
```

```
  "brake": 0,
  "accel_x": 0.72,
  "accel_y": 0.91,
  "yaw_rate": 18.4,
  "engine_temp": 81.5,
  "distance": 14.2
}
```

On-device, use a compact binary or CSV-style record for high-frequency logging — convert to JSON/CSV/Parquet later during post-run analysis, not on the microcontroller itself.

### Telemetry communication — what goes to the pit

The kart sends a small live packet, not the full dataset:

- timestamp
- speed
- RPM
- gear
- engine temperature
- steering
- G-force
- time-to-contact (TTC)
- warning flag

The complete dataset stays on the SD card for post-run Python analysis; the pit only ever receives the filtered live subset.

### Driver display — keep it minimal

The driver needs actionable information only, not a full data dump:

```
+-----------------------+
|       KURUKSHETRA     |
|                       |
|        67 km/h        |
|                       |
| RPM     7240          |
| GEAR       4          |
|                       |
| ENGINE     82C        |
|                       |
|        TTC  2.4s      |
|        CAUTION        |
+-----------------------+
```

## 3.2 Pit Software

The pit receives the live telemetry packet and turns it into a dashboard.

| Layer | Recommended technology |
|---|---|
| Kart firmware | C/C++ — ESP-IDF or Arduino framework |
| MCU | ESP32 |
| Data logging | microSD |
| Wireless | ESP-NOW or suitable RF link |
| Driver display | OLED / TFT |
| Pit receiver | ESP32 or laptop |
| Backend | Python + FastAPI |
| Dashboard frontend | React (or Streamlit for fastest prototype) |

| Layer | Recommended technology |
|---|---|
| Database | SQLite (V1) → PostgreSQL (later) |
| Analytics | Python + Pandas + NumPy |
| Scientific computing | SciPy |
| Visualization | Plotly |
| Track processing | GeoPandas / Shapely |
| Simulation V1 | Custom Python physics model |
| Simulation V2 | Project Chrono (stretch goal) |
| Version control | Git + GitHub |

### Pit dashboard layout — four areas

| Area | Fields shown |
|---|---|
| Vehicle | Speed, RPM, gear, throttle % |
| Dynamics | Lateral G, longitudinal G, steering angle, yaw rate |
| Health | Engine temperature, system voltage, sensor status |
| Safety | Time-to-contact, obstruction detected, status (safe/caution/danger) |

## 3.3 Data & Analytics Pipeline

Once SD-card data comes off the kart, it moves through a defined pipeline before any analysis happens:

```
RAW DATA
   |
VALIDATION
   |
CLEANING
   |
TIMESTAMP SYNC
   |
FILTERING
   |
FEATURE EXTRACTION
   |
ANALYSIS
```

### Automatic lap and sector analysis

Rather than manually reading graphs, the pipeline should automatically identify start/finish, braking zones, corner entry, apex, corner exit, and acceleration zones — then compare laps and sectors directly:

| | Lap 1 | Lap 2 | Lap 3 |
|---|---|---|---|
| Sector 1 | 31.2 | 30.9 | 30.5 |
| Sector 2 | 29.8 | 29.4 | 28.9 |
| Sector 3 | 31.4 | 30.5 | 30.6 |

This is far more useful than “lap 3 was faster” — it shows exactly where the improvement came from, which can then be cross-referenced against speed, brake, steering, and throttle data for that sector.

## 3.4 Database Structure

Store data as sessions, not one giant table:

```
DATABASE
|
```

```
+-- Drivers
+-- Vehicles
+-- Tracks
+-- Sessions
+-- Telemetry
+-- Weather
+-- Surface Conditions
+-- Simulation Results
```

Each session record carries its own metadata (track, date, driver, temperature, surface, kart setup) linked to its telemetry, weather, and results — this becomes valuable once dozens of sessions accumulate.

## 3.5 Recommended Build Order

| Phase | Deliverable |
|---|---|
| 1 | ESP32 + sensors + SD logging |
| 2 | Steering display + obstruction warning |
| 3 | Wireless telemetry + pit dashboard |
| 4 | Python data analysis pipeline |
| 5 | Track reconstruction + database |
| 6 | Kart physics model |
| 7 | Digital twin (see Section 4) |
| 8 | Weather / surface simulation |
| 9 | Optimization / what-if simulation |

*Do not build this simultaneously across all phases — each phase depends on the previous one producing real, validated data.*

# 4. Digital Twin

The digital twin is the most technically demanding part of the project — and the strongest differentiator for the innovation round, provided it's built honestly. It combines four separate models into one simulator.

## 4.1 The Four Models

| Model | Represents | Data source |
|---|---|---|
| Track model | Centerline, curvature, segment types, elevation | GPS + track-position telemetry from real laps |
| Kart model | Mass, wheelbase, CG, torque curve, gear ratios, tire grip, drag | Partly measured, partly spec sheet, partly calibrated against real telemetry |
| Environment model | Temperature, humidity, wind, surface grip | Weather API/dataset or live BME280 readings |
| Driver behaviour | Throttle/brake/steering patterns per corner | Extracted directly from telemetry logs |

## 4.2 Honest Difficulty Assessment

### Track model — achievable

GPS position, speed, and steering angle from real laps is enough to reconstruct a centerline and classify segments (straight / corner / braking zone). This is mostly geometry, buildable with GeoPandas/Shapely.

### Kart model — the hard part

Mass, wheelbase, and gear ratios are simple measurements or spec lookups. But tire grip, rolling resistance, and drag coefficient cannot be looked up — they must be back-calculated by comparing simulated behaviour against real telemetry and iterating:

```
Guess tire grip value
       |
Run simulation
       |
Compare simulated corner speed vs real telemetry corner speed
       |
Adjust grip value
       |
Repeat until simulation matches reality within acceptable error
```

> **This calibration loop is the project's real differentiator**
>
> It is genuinely research-grade work condensed into a student project. This is the part to present explicitly to judges — it is the strongest evidence of engineering rigor, and skipping it while still calling the result a “digital twin” is the biggest credibility risk in the whole project.

### Environment model — straightforward

Weather API or live BME280 readings feed in as parameters directly; no significant complexity.

### Physics engine — scope-dependent

A basic point-mass model (F = ma, simple friction-circle tire model, no suspension dynamics) is realistic to build from scratch in Python within the project timeline. A full multi-body vehicle dynamics engine (e.g. Project Chrono) is a serious undertaking better suited as a stated V2/stretch direction, not a V1 deliverable.

## 4.3 Recommended V1 Digital Twin — Point-Mass Simulator

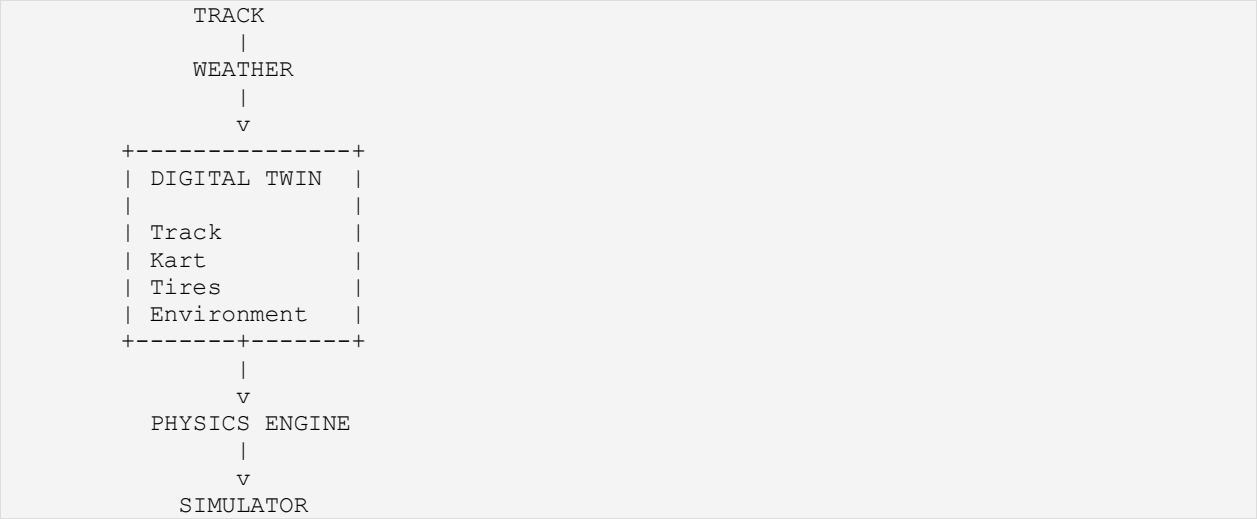
Rather than a full vehicle-dynamics engine, build a point-mass simulator:

- Track represented as a sequence of segments: straight, corner (with radius), braking zone
- Kart represented as: mass, max lateral G (from tire grip estimate), engine power curve, drag coefficient

- At each segment, compute max achievable speed given available grip and power, then integrate to get lap time
- Calibrate against 2–3 real laps; adjust tire grip and drag until predicted lap time is within roughly 5% of actual

**This is honest, buildable in the available timeline, and supports a much stronger claim than "we built a simulator" — it supports "we validated our simulator against real telemetry."**

## 4.4 Digital Twin Data Flow




The simulator receives track, kart, driver, weather, and surface parameters, and produces speed, acceleration, G-force, lap time, and energy/fuel-use predictions.

## 4.5 What-If Simulation (Stretch)

Once the simulator is calibrated, individual variables can be swept to study their effect:

| Variable | Example values |
|---|---|
| Speed | 45 / 50 / 55 / 60 / 65 km/h |
| Surface | Dry / damp / wet / low grip |
| Temperature | 25°C / 30°C / 35°C / 40°C / 45°C |
| Wind | 0 / 10 / 20 / 30 km/h |

Optimization — comparing multiple full setups against each other to find the best predicted configuration — comes last, only after the core simulator is validated. Do not attempt optimization before validation.

## 5. Key Risks Across the Whole System

### Data volume vs. wireless bandwidth

A kart has no Wi-Fi range or LTE on a typical track. Stream only a small live subset (speed, RPM, obstruction warning, engine temp); log everything else to SD card for post-session review.

### Engine electrical noise and vibration

The most common failure point in student telemetry builds is wiring and electrical isolation near the engine, not code. Use shielded wiring, vibration-rated mounts, and isolate the ignition system from the microcontroller supply.

### Processing load

Reading many sensors, computing derived values, driving a display, and handling wireless transmission all at once needs the headroom of an ESP32 (dual-core), not a basic Arduino Nano.

### Scope creep

The single biggest risk is not technical — it's committing to the full V3 sensor and digital-twin scope before the V1 core system is working reliably. Lock the phased build order in Section 3.5 before Day 1, and treat later phases as stretch goals until each prior phase is validated.

### Digital twin credibility

Presenting a simulator without validating it against real telemetry is the biggest credibility risk in the project. Always be able to show the calibration comparison — predicted vs. actual lap time or corner speed — rather than just the simulator output alone.

# 6. Innovation Round Pitch

**One-line pitch**

We don't just collect telemetry — we convert real vehicle data into a validated computational model, and use that model to test conditions that are expensive or impossible to reproduce physically.

## Anticipated questions

| Likely question | Answer |
|---|---|
| Why not stream everything live? | Bandwidth and reliability — explain the live-vs-logged split; even advanced reference builds use exactly this pattern |
| What's your actual data latency? | State the number measured during testing, not an estimate |
| How do you handle engine electrical noise? | Explain the shielding/isolation approach directly — shows understanding of the real engineering risk |
| Why wheel-based speed and not GPS? | GPS isn't accurate enough on tight kart-track turns; wheel-based sensing is standard at this track scale |
| How do you know your simulator is accurate? | Show the calibration comparison — predicted vs. actual lap time / corner speed from real runs |
| What happens if a sensor fails mid-run? | System continues logging other channels; driver display shows a fault indicator rather than failing silently |